

DESIGN AND ANALYSIS OF ALGORITHMS

CS-551 MJ | F.Y. M.Sc. Computer Science | Semester II
Complete Step-by-Step Study Guide | 2023 Credit Pattern

Prepared by: Aditya Bapusaheb Chaudhari, Full Stack Engineer
Contact Details

LinkedIn: <https://www.linkedin.com/in/aditya-chaudhari0301/>

GitHub: <https://github.com/chaudhari-aditya03>

Email: aditya.chaudhari.dev@gmail.com

EXAM WEIGHTAGE

Q1: 10 marks (2×5) | Q2–Q5: 12 marks each (6×2) | Q6: 12 marks (4×3) | TOTAL: 70 marks

UNIT 1: ANALYSIS OF ALGORITHMS

1.1 Time Complexity & Space Complexity

Time Complexity: The amount of time an algorithm takes to run as a function of the input size n . It counts the number of basic operations executed.

Space Complexity: The amount of memory an algorithm needs as a function of input size n . It includes both auxiliary space and space for input.

1.2 Asymptotic Notations (MOST IMPORTANT - Always in Q1)

Asymptotic notations describe the behavior of an algorithm's time/space complexity as $n \rightarrow \infty$.

Notation	Name	Meaning	Example
$O(f(n))$	Big-O	Upper bound (worst case). $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$	$O(n^2)$ for bubble sort
$\Omega(f(n))$	Big-Omega	Lower bound (best case). $T(n) \geq c \cdot f(n)$ for all $n \geq n_0$	$\Omega(n \log n)$ for comparison sort
$\Theta(f(n))$	Big-Theta	Tight bound (exact order). Both upper and lower bound	$\Theta(n \log n)$ for merge sort
$o(f(n))$	Little-o	Strict upper bound. $T(n) < c \cdot f(n)$ for all $c > 0$	$n = o(n^2)$
$\omega(f(n))$	Little-omega	Strict lower bound. $T(n) > c \cdot f(n)$ for all $c > 0$	$n^2 = \omega(n)$

KEY EXAM TIP

Order of growth (slowest to fastest): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Complexity	Name	$n=10$	$n=100$	Speed
$O(1)$	Constant	1	1	⚡ Fastest
$O(\log n)$	Logarithmic	3.32	6.64	⚡ Very Fast
$O(n)$	Linear	10	100	✓ Fast
$O(n \log n)$	Linearithmic	33	664	✓ Good
$O(n^2)$	Quadratic	100	10,000	⚡ Slow
$O(n^3)$	Cubic	1000	1,000,000	✗ Very Slow

$O(2^n)$	Exponential	1024	Huge	✗ Terrible
----------	-------------	------	------	------------

Formal Definitions:

Big-O: $T(n) = O(f(n))$ iff $\exists c > 0, n_0 \geq 1$ such that $T(n) \leq c \cdot f(n) \forall n \geq n_0$

Omega: $T(n) = \Omega(f(n))$ iff $\exists c > 0, n_0 \geq 1$ such that $T(n) \geq c \cdot f(n) \forall n \geq n_0$

Theta: $T(n) = \Theta(f(n))$ iff $T(n) = O(f(n))$ AND $T(n) = \Omega(f(n))$

1.3 Divide and Conquer Strategy

Definition: A paradigm that breaks a problem into smaller subproblems, solves them recursively, and combines results.

Control Abstraction for Divide & Conquer

DAndC (P) :

```

if Small(P) then return S(P)
else
  divide P into P1, P2, ..., Pk (k ≥ 1)
  apply DAndC to each subproblem
  return Combine(DAndC(P1), DAndC(P2), ..., DAndC(Pk))

```

Three Steps: 1) Divide – Split problem 2) Conquer – Solve recursively 3) Combine – Merge solutions

Examples: Binary Search, Merge Sort, Quick Sort, Finding Maximum/Minimum, Strassen's Matrix Multiplication

UNIT 2: SORTING ALGORITHMS

2.1 Insertion Sort (Step-by-Step)

Idea: Build sorted array one element at a time by inserting each element into its correct position.

Time Complexity: Best: $O(n)$ | Average: $O(n^2)$ | Worst: $O(n^2)$

Space: $O(1)$ — In-place sorting

Algorithm:

Insertion Sort Algorithm

```
for i = 1 to n-1 do
    key = A[i]
    j = i - 1
    while j >= 0 AND A[j] > key do
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key
```

Example: Sort 15, 7, 30, 5, 12

Pass	Key	Array State
Initial	-	15, 7, 30, 5, 12
i=1	7	7, 15, 30, 5, 12
i=2	30	7, 15, 30, 5, 12
i=3	5	5, 7, 15, 30, 12
i=4	12	5, 7, 12, 15, 30 ✓

2.2 Merge Sort (Step-by-Step)

Idea: Divide array into halves, sort each half recursively, then merge sorted halves.

Time Complexity: Best/Average/Worst: $O(n \log n)$ | Space: $O(n)$

NOTE: **Merge Sort is NOT in-place!** It requires auxiliary array. Contrary to what some might say.

Merge Sort Algorithm

```
MergeSort(A, low, high):
    if low < high then
        mid = (low + high) / 2
        MergeSort(A, low, mid)
        MergeSort(A, mid+1, high)
        Merge(A, low, mid, high)
```

Example: Sort 26, 5, 37, 1, 61, 11, 59, 15

Step 1 — Divide:

[26, 5, 37, 1] | [61, 11, 59, 15]
[26, 5] | [37, 1] | [61, 11] | [59, 15]
[26] | [5] | [37] | [1] | [61] | [11] | [59] | [15]

Step 2 — Merge back:

[5, 26] | [1, 37] | [11, 61] | [15, 59]
[1, 5, 26, 37] | [11, 15, 59, 61]
[1, 5, 11, 15, 26, 37, 59, 61] ✓

2.3 Quick Sort (Step-by-Step)

Idea: Pick a pivot, partition array so elements < pivot are left, elements > pivot are right, recurse.

Time Complexity: Best/Average: $O(n \log n)$ | Worst: $O(n^2)$ (when pivot is always smallest/largest)

Best Case Time Complexity Derivation

Best case: pivot divides array into two equal halves each time.

$T(n) = 2T(n/2) + O(n)$ [2 subproblems of size $n/2$, partition takes $O(n)$]

By Master Theorem: $T(n) = O(n \log n)$

Example: Sort 18, 25, 34, 28, 9, 10, 7 (pivot = first element = 18)

Pass 1: Pivot = 18. Elements < 18: {9, 10, 7}. Elements > 18: {25, 34, 28}.

[7, 9, 10, 18, 25, 34, 28]

Pass 2L: Sort [7, 9, 10], pivot=7 → [7, 9, 10] ✓

Pass 2R: Sort [25, 34, 28], pivot=25 → [25, 28, 34] ✓

Final: [7, 9, 10, 18, 25, 28, 34] ✓

UNIT 3: GREEDY METHOD

3.1 Key Terms

Feasible Solution: A solution that satisfies all given constraints of the problem.

Optimal Solution: A feasible solution that maximizes (or minimizes) the objective function — the best among all feasible solutions.

Greedy Strategy: Make the locally optimal choice at each step hoping to find a global optimum. No backtracking.

3.2 Fractional Knapsack (Greedy)

Problem: Given n items with profits $P[]$ and weights $W[]$, knapsack capacity m . Maximize total profit. Items can be broken (fractional).

Greedy Strategy for Knapsack

Sort items by profit/weight ratio in decreasing order. Pick items greedily — take full item if it fits, else take fraction to fill remaining capacity.

Example (from exam): $n=7, m=15$

$P = (10, 5, 15, 7, 6, 18, 3), W = (2, 3, 5, 7, 1, 4, 1)$

Item	P	W	P/W	Selected?	Amount	Profit
6	18	4	4.50	✓ Full	4	18
1	10	2	5.00	✓ Full	2	10
3	15	5	3.00	✓ Full	5	15
5	6	1	6.00	✓ Full	1	6
7	3	1	3.00	✓ Full	1	3
2	5	3	1.67	✗ 2/3	2	~3.33

Total Profit ≈ 55.33 | Capacity used = 15 ✓

3.3 Job Sequencing with Deadlines (Greedy)

Problem: n jobs, each with profit p_i and deadline d_i . Each job takes 1 unit time. Maximize total profit by selecting a subset of jobs.

Greedy Strategy

Sort jobs by profit in decreasing order. Schedule each job in the latest possible free slot before its deadline.

Example (from exam): $n=5$, $P=(10,8,15,5,7)$, $d=(4,1,3,2,3)$

Step 1 — Sort by profit (decreasing):

Job	j3	j1	j2	j5	j4
Profit	15	10	8	7	5
Deadline	3	4	1	3	2

Step 2 — Schedule jobs (slots 1 to 4):

- j3 ($d=3$, $p=15$): Place in slot 3 → Slots: [-, -, j3, -]
- j1 ($d=4$, $p=10$): Place in slot 4 → Slots: [-, -, j3, j1]
- j2 ($d=1$, $p=8$): Place in slot 1 → Slots: [j2, -, j3, j1]
- j5 ($d=3$, $p=7$): Slot 3 taken, slot 2 free → Slots: [j2, j5, j3, j1]
- j4 ($d=2$, $p=5$): Slots 1,2 taken → Skipped ✗

✓ Optimal Solution

Jobs selected: j1, j2, j3, j5 | Sequence: [j2, j5, j3, j1] | Total Profit = $10+8+15+7 = 40$

3.4 MRT (Minimum Retrieval Time) — Tape Storage

Problem: n programs stored on tape. MRT is minimized by storing programs in increasing order of length.

Example: Programs with lengths 14, 5, 20, 8, 3

Step 1 — Sort ascending: 3, 5, 8, 14, 20

Step 2 — Calculate retrieval times:

$$t_1=3, \quad t_2=3+5=8, \quad t_3=3+5+8=16, \quad t_4=3+5+8+14=30, \quad t_5=3+5+8+14+20=50$$

$$\text{MRT} = (3 + 8 + 16 + 30 + 50) / 5 = 107/5 = 21.4$$

3.5 Kruskal's Algorithm (Minimum Spanning Tree)

Idea: Sort all edges by weight. Add edge to MST if it doesn't form a cycle (use Union-Find). Stop when MST has $n-1$ edges.

Kruskal's Algorithm Steps

1. Sort all edges in ascending order of weight.
2. Initialize each vertex as its own component.
3. For each edge (u,v) in sorted order:
 - If u and v are in different components → Add edge, union components.
 - Else → Skip (would form cycle).
4. Repeat until MST has (n-1) edges.

Example: Graph with edges A-B:9, A-C:16, A-D:5, B-D:7, B-E:8, C-D:12, C-F:4, D-E:6, D-F:14, D-G:13, E-G:10, F-G:11

Edges sorted: CF:4, AD:5, DE:6, BD:7, BE:8, AB:9, EG:10, FG:11, CD:12, DG:13, DF:14, AC:16

- CF:4 → Add ✓ (C-F)
- AD:5 → Add ✓ (A-D)
- DE:6 → Add ✓ (D-E)
- BD:7 → Add ✓ (B-D, connects B to tree)
- BE:8 → Skip ✗ (B and E already connected via D)
- AB:9 → Skip ✗ (A and B already connected via D)
- EG:10 → Add ✓ (G connected)
- FG:11 → Skip ✗ (F and G already connected)
- CD:12 → Add ✓ (C connected to main tree)

MST Edges: CF, AD, DE, BD, EG, CD | Total Weight = 4+5+6+7+10+12 = 44

3.6 Prim's Algorithm (MST)

Idea: Start from any vertex. Always add the minimum weight edge connecting the MST to a non-MST vertex.

Prim's vs Kruskal's

Prim's: grows one tree from a source — good for dense graphs. Kruskal's: picks globally minimum edges — good for sparse graphs. Both give same MST cost.

UNIT 4: DYNAMIC PROGRAMMING

4.1 Principle of Optimality

Definition: An optimal solution to any instance of a problem always contains optimal solutions to all sub-instances. If this principle holds, we can use Dynamic Programming.

Example: Shortest path: if the shortest path from A→C goes through B, then A→B must also be the shortest path from A to B.

4.2 Longest Common Subsequence (LCS)

Problem: Find the longest subsequence common to two strings X and Y (characters need not be contiguous).

LCS Recurrence Relation

$$\begin{aligned} c[i][j] &= 0 && \text{if } i=0 \text{ or } j=0 \\ c[i][j] &= c[i-1][j-1] + 1 && \text{if } X[i] = Y[j] \\ c[i][j] &= \max(c[i-1][j], c[i][j-1]) && \text{if } X[i] \neq Y[j] \end{aligned}$$

Example: X = <A,C,D,A,C,D>, Y = <C,B,C,D,C>

Build the DP table (rows = X characters, columns = Y characters):

		C	B	C	D	C
		0	0	0	0	0
A	A	0	0	0	0	0
C	C	1	1	1	1	1
D	D	1	1	1	2	2
A	A	1	1	1	2	2
C	C	1	1	2	2	3
D	D	1	1	2	3	3

✓ LCS Length = 3. Trace back to find: LCS = <C,D,C> or <C,C,D>

To find LCS string: start at $c[m][n]$, if $X[i]=Y[j] \rightarrow$ take character & go diagonal, else go to $\max(\text{up}, \text{left})$.

Example 2: X=<A,B,C,B,D,A,B>, Y=<B,D,C,A,B,A>

Answer: LCS = <B,C,B,A> or <B,D,A,B>, Length = 4

4.3 Matrix Chain Multiplication

Problem: Find the optimal parenthesization of matrix chain $A_1 \times A_2 \times \dots \times A_n$ to minimize scalar multiplications.

Recurrence Relation

$$m[i][i] = 0$$
$$m[i][j] = \min \text{ over all } k (i \leq k < j) \text{ of:}$$
$$m[i][k] + m[k+1][j] + p[i-1] \times p[k] \times p[j]$$

Example: $A_1(10 \times 4)$, $A_2(4 \times 7)$, $A_3(7 \times 5)$, $A_4(5 \times 6)$

Dimensions array $p = [10, 4, 7, 5, 6]$

Single matrices (L=1): $m[1][1] = m[2][2] = m[3][3] = m[4][4] = 0$

L=2 (pairs):

- $m[1][2] = p[0] \times p[1] \times p[2] = 10 \times 4 \times 7 = 280$
- $m[2][3] = p[1] \times p[2] \times p[3] = 4 \times 7 \times 5 = 140$
- $m[3][4] = p[2] \times p[3] \times p[4] = 7 \times 5 \times 6 = 210$

L=3 (triples):

- $m[1][3]$: $k=1$: $m[1][1] + m[2][3] + 10 \times 4 \times 5 = 0 + 140 + 200 = 340$; $k=2$:
 $m[1][2] + m[3][3] + 10 \times 7 \times 5 = 280 + 0 + 350 = 630 \rightarrow \min = 340$
- $m[2][4]$: $k=2$: $m[2][2] + m[3][4] + 4 \times 7 \times 6 = 0 + 210 + 168 = 378$; $k=3$:
 $m[2][3] + m[4][4] + 4 \times 5 \times 6 = 140 + 0 + 120 = 260 \rightarrow \min = 260$

L=4 (full chain):

- $m[1][4]$: $k=1$: $m[1][1] + m[2][4] + 10 \times 4 \times 6 = 0 + 260 + 240 = 500$
- $k=2$: $m[1][2] + m[3][4] + 10 \times 7 \times 6 = 280 + 210 + 420 = 910$
- $k=3$: $m[1][3] + m[4][4] + 10 \times 5 \times 6 = 340 + 0 + 300 = 640 \rightarrow \min = 500$

✓ Minimum multiplications = 500. Optimal: $((A_1)(A_2A_3A_4)) \rightarrow A_1 \times (A_2 \times (A_3 \times A_4))$

The optimal parenthesization is $A_1 \times (A_2 \times (A_3 \times A_4))$ with minimum cost 500 scalar multiplications.

4.4 Travelling Salesperson Problem (TSP) — Dynamic Programming

Problem: Find the shortest Hamiltonian circuit (visits all cities exactly once and returns to start).

TSP Recurrence (Held-Karp)

$g(i, S) = \min \text{ over } j \in S \text{ of } \{ c[i][j] + g(j, S - \{j\}) \}$
where S = set of unvisited cities, i = current city, $c[i][j]$ = cost from i to j
Base case: $g(i, \{ \}) = c[i][1]$ (return to city 1)

Example: Cost matrix (from exam):

$A = [\infty, 10, 8, 14 / 22, \infty, 7, 6 / 9, 10, \infty, 4 / 5, 6, 2, \infty]$ (cities 1,2,3,4)

Step 1 — Base cases (return to city 1):

$g(2, \emptyset) = c[2][1] = 22$, $g(3, \emptyset) = c[3][1] = 9$, $g(4, \emptyset) = c[4][1] = 5$

Step 2 — Sets of size 1:

$g(1, \{2\}) = c[1][2] + g(2, \emptyset) = 10 + 22 = 32$

$g(1, \{3\}) = c[1][3] + g(3, \emptyset) = 8 + 9 = 17$

$g(1, \{4\}) = c[1][4] + g(4, \emptyset) = 14 + 5 = 19$

$g(2, \{3\}) = c[2][3] + g(3, \emptyset) = 7 + 9 = 16$

$g(2, \{4\}) = c[2][4] + g(4, \emptyset) = 6 + 5 = 11$

$g(3, \{2\}) = c[3][2] + g(2, \emptyset) = 10 + 22 = 32$

$g(3, \{4\}) = c[3][4] + g(4, \emptyset) = 4 + 5 = 9$

$g(4, \{2\}) = c[4][2] + g(2, \emptyset) = 6 + 22 = 28$

$g(4, \{3\}) = c[4][3] + g(3, \emptyset) = 2 + 9 = 11$

Step 3 — Sets of size 2:

$g(2, \{3, 4\}) = \min(c[2][3] + g(3, \{4\}), c[2][4] + g(4, \{3\})) = \min(7 + 9, 6 + 11) = \min(16, 17) = 16$

$g(3, \{2, 4\}) = \min(c[3][2] + g(2, \{4\}), c[3][4] + g(4, \{2\})) = \min(10 + 11, 4 + 28) = \min(21, 32) = 21$

$g(4, \{2, 3\}) = \min(c[4][2] + g(2, \{3\}), c[4][3] + g(3, \{2\})) = \min(6 + 16, 2 + 32) = \min(22, 34) = 22$

Step 4 — Full solution (start at city 1):

$g(1, \{2, 3, 4\}) = \min(c[1][2] + g(2, \{3, 4\}), c[1][3] + g(3, \{2, 4\}), c[1][4] + g(4, \{2, 3\}))$
 $= \min(10 + 16, 8 + 21, 14 + 22) = \min(26, 29, 36) = 26$

✓ Minimum TSP cost = 26. Optimal tour: 1→2→3→4→1

The Held-Karp dynamic programming approach gives optimal TSP tour of cost 26.

4.5 Floyd-Warshall Algorithm (All-Pairs Shortest Path)

Purpose: Find shortest paths between all pairs of vertices, handles negative weights (but not negative cycles).

Recurrence Relation

$D^0[i][j] = A[i][j]$ (initial distances)

$$D^k[i][j] = \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j])$$

Repeat for $k = 1$ to n . Time Complexity: $O(n^3)$

Negative Weighted Cycle: A cycle where total weight is negative. Floyd-Warshall detects it ($D[i][i] < 0$) but cannot give correct shortest paths.

4.6 Bellman-Ford Algorithm

Purpose: Single-source shortest paths. Works with negative edge weights. Detects negative cycles.

Bellman-Ford Algorithm

Initialize: $\text{dist}[\text{source}] = 0$, $\text{dist}[\text{all others}] = \infty$

Repeat $(n-1)$ times:

For each edge (u, v, w) :

if $\text{dist}[u] + w < \text{dist}[v]$: $\text{dist}[v] = \text{dist}[u] + w$

Check for negative cycles: if any edge still relaxable $\rightarrow$ negative cycle exists

4.7 String Editing Problem

Problem: Transform string X into Y using Insert(cost 1), Delete(cost 1), Interchange/Replace(cost 2). Find minimum cost.

Explanation / Steps for DP Table Construction:

- Initialization:** Fill the first row ($E[0][j]$) and first column ($E[i][0]$).
 - $E[i][0] = i$ (cost to delete i characters from X).
 - $E[0][j] = j$ (cost to insert j characters to form Y).
- Fill Remaining Cells:** For each cell $E[i][j]$, compare $X[i]$ and $Y[j]$.
 - If $X[i] = Y[j]$: Cost is $E[i-1][j-1]$ (match/no cost) + 0.
 - If $X[i] \neq Y[j]$: Cost is the minimum of three possibilities:
 - Deletion: $E[i-1][j] + 1$
 - Insertion: $E[i][j-1] + 1$
 - Interchange/Replace: $E[i-1][j-1] + 2$ (since the problem states interchange cost is 2).
- Final Result:** The minimum edit distance is $E[m][n]$, where m is the length of X and n is the length of Y.

Recurrence Relation

$E[0][j] = j$ (j insertions to build Y from empty)

$E[i][0] = i$ (i deletions to empty X)

$E[i][j] = \min$ of:

$E[i-1][j] + 1$ (delete from X)

$E[i][j-1] + 1$ (insert into X)

$E[i-1][j-1] + 0$ (if $X[i]=Y[j]$, no cost)

$E[i-1][j-1] + 2$ (if $X[i] \neq Y[j]$, interchange)

Example: X=AABABA, Y=BAABA

Build 7x6 table (rows=X, cols=Y)

		B	A	A	B	A	
		0	1	2	3	4	5
A	A	1	2	1	2	3	4
A	A	2	3	2	1	2	3
B	B	3	2	3	2	1	2
A	A	4	3	2	3	2	1
B	B	5	4	3	4	3	2
A	A	6	5	4	3	4	3

□ Minimum Edit Distance (cost) = $E[6][5] = 3$

It takes 3 edit operations (insert/delete/interchange) to transform X=AABABA into Y=BAABA.

UNIT 5: GRAPH ALGORITHMS

5.1 BFS and DFS

BFS (Breadth First Search): Explores layer by layer using a Queue. Finds shortest path in unweighted graphs.

DFS (Depth First Search): Explores as deep as possible using a Stack (or recursion).

BFS Algorithm	DFS Algorithm
<pre>Enqueue start; mark visited while Queue not empty: v = Dequeue() visit v for each neighbor u: if not visited: Enqueue(u)</pre>	<pre>DFS(v): mark v as visited visit v for each neighbor u: if not visited: DFS(u)</pre>

5.2 Types of Edges in DFS

Edge Type	Definition
Tree Edge	Edge in the DFS tree. (u,v) where v was first discovered through u .
Back Edge	Edge from a descendant to an ancestor in DFS tree. Indicates a cycle. (u,v) where v is ancestor of u .
Forward Edge	Non-tree edge from ancestor to descendant. (u,v) where v is descendant of u but not child.
Cross Edge	Edge between vertices in different subtrees or between two DFS trees. (u,v) where neither is ancestor of the other.

5.3 Strongly Connected Components (SCC)

Definition: A maximal set of vertices in a directed graph such that there is a path from each vertex to every other vertex in the set.

Algorithm to Find SCCs (Kosaraju's)

1. Run DFS on original graph G , record finish times.
2. Transpose the graph G (reverse all edges).
3. Run DFS on transposed graph in decreasing order of finish time.
4. Each DFS tree in step 3 = one SCC.

5.4 Dijkstra's Algorithm

Purpose: Single-source shortest path in graphs with NON-NEGATIVE edge weights.

Dijkstra vs Bellman-Ford

Dijkstra: $O(V^2)$ or $O(E \log V)$ with heap. Cannot handle negative weights.
Bellman-Ford: $O(VE)$. Handles negative weights. Detects negative cycles.

5.5 Articulation Points

Definition: A vertex in an undirected graph whose removal increases the number of connected components. Found using DFS by checking low values.

UNIT 6: BACKTRACKING

6.1 N-Queens Problem

Problem: Place N queens on N×N chessboard such that no two queens threaten each other (no same row, column, or diagonal).

4-Queens Problem (Complete Solution):

State space tree: try each column for each row. Backtrack if constraint violated.

All Solutions for 4-Queens

Solution 1: Q at (1,2), (2,4), (3,1), (4,3) → [2,4,1,3]

Solution 2: Q at (1,3), (2,1), (3,4), (4,2) → [3,1,4,2]

Total: 2 fundamental solutions (92 total with rotations/reflections for 8-queens)

Constraint check: For queens at positions (r1,c1) and (r2,c2): $c1 \neq c2$ AND $|r1-r2| \neq |c1-c2|$

6.2 m-Coloring Problem

Problem: Color the vertices of a graph using at most m colors such that no two adjacent vertices have the same color.

Backtracking Approach

1. Assign color 1 to vertex 1.
2. For each vertex v (1 to n):
Try colors 1 to m. A color c is valid if no adjacent vertex has color c.
3. If no valid color → backtrack.
4. If all vertices colored → record solution.

Graph: Vertices 1-5, Edges: 1-2, 1-4, 2-3, 2-5, 3-5, 4-5. Colors: 1,2,3

Adjacency: 1→{2,4}, 2→{1,3,5}, 3→{2,5}, 4→{1,5}, 5→{2,3,4}

Some valid 3-colorings:

- V1=1, V2=2, V3=1, V4=2, V5=3
- V1=1, V2=2, V3=3, V4=3, V5=1 (verify: no adjacent vertices share color)
- V1=2, V2=1, V3=2, V4=3, V5=3 — check adjacency carefully

6.3 Hamiltonian Cycle

Definition: A Hamiltonian cycle visits every vertex exactly once and returns to the start vertex.

Difference from Euler circuit: Euler visits every EDGE once; Hamiltonian visits every VERTEX once.

For graph 1-2-3-4 (complete K_4)

Hamiltonian cycles: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$, $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$, $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$

Note: Clockwise = Counterclockwise, so distinct cycles = $(n-1)!/2$

UNIT 7: BRANCH AND BOUND

7.1 Key Terms

Live Node: A node that has been generated but whose children have not yet been generated.

Dead Node: A node that is not to be expanded further — either all children have been generated, or it has been killed by a bound.

E-Node: (Expansion Node) The current live node being expanded.

7.2 Branch and Bound Strategy

Definition

An algorithm design technique that systematically enumerates candidate solutions by means of state space search, using upper/lower bounds to prune branches that cannot yield better solutions than the current best.

Type	Node Selection	Use Case
FIFO B&B	First In First Out (Queue)	BFS order exploration
LIFO B&B	Last In First Out (Stack)	DFS order exploration
LCBB (Least Cost)	Node with minimum cost bound	Most efficient, like TSP

7.3 0/1 Knapsack using LCBB

Problem: Items cannot be broken (unlike fractional knapsack). Either take the full item or don't.

Example: $n=4$, $P=(10,10,12,18)$, $W=(2,4,6,9)$, $m=15$

Step 1 — Compute p/w ratios and sort: Item4($18/9=2.0$), Item1($10/2=5.0$), Item2($10/4=2.5$), Item3($12/6=2.0$)

Sorted order: Item1(5.0), Item2(2.5), Item3(2.0), Item4(2.0)

Upper bound calculation: Use fractional knapsack to compute upper bound at each node.

Node 0 (root): $UB = 10 + 10 + 12 + (15-12) \times 18/9 = 10+10+12+6 = 38$

Branch: Include Item1 ($W=2$, $P=10$, remaining=13)

Branch: Exclude Item1

✓ Best solution found: Include items 1,2,4 → $P=10+10+18=38$, $W=2+4+9=15$ ✓

Using LCBB, the optimal 0/1 knapsack solution selects items 1, 2, and 4 for maximum profit of 38.

UNIT 8: NP CLASS & BEYOND

8.1 P and NP Classes

P Class	NP Class
Problems solvable in POLYNOMIAL time $O(n^k)$ by a deterministic algorithm. Examples: Sorting, Shortest path, MST, Binary Search	Problems where a given solution can be VERIFIED in polynomial time. May not be solvable in polynomial time. Examples: TSP, Hamiltonian cycle, Graph coloring, Satisfiability

Key Fact

$P \subseteq NP$. Whether $P = NP$ is the greatest unsolved problem in computer science (Millennium Problem). NP-Complete = hardest problems in NP.

8.2 Non-Deterministic Algorithms

Definition: An algorithm that can choose any one of a set of possible computations at each step. Uses two special operations:

- **choice(S):** Arbitrarily selects one element from set S.
- **success / failure:** Signals successful or unsuccessful completion.

Non-Deterministic Sort Algorithm

```
Algorithm NDSort(A, n):
  // Generate a permutation
  for i = 1 to n:
    B[i] = choice(1, n)
  if B is not a permutation: failure
  // Check if sorted
  for i = 1 to n-1:
    if B[i] > B[i+1]: failure
  A = B; success
```

8.3 Satisfiability Problem (SAT)

Definition: Given a Boolean formula in CNF (Conjunctive Normal Form), determine if there exists an assignment of truth values that makes the formula true.

Significance: SAT was the first problem proven to be NP-Complete (Cook's theorem, 1971).

UNIT 9: HUFFMAN CODING

9.1 Huffman Coding

Purpose: Optimal prefix-free binary encoding. Frequently occurring characters get shorter codes, rare ones get longer codes.

Huffman Algorithm

1. Create a leaf node for each character, insert into Min-Heap.
2. While heap has more than 1 node:
 - a. Extract two minimum frequency nodes x and y .
 - b. Create new node z with $\text{freq} = \text{freq}(x) + \text{freq}(y)$.
 - c. $z.\text{left} = x$, $z.\text{right} = y$. Insert z into heap.
3. Remaining node is root. Assign 0 to left edges, 1 to right edges.

Time Complexity: $O(n \log n)$

Example: Frequencies 6, 7, 9, 10, 12, 14, 22

Step 1 — Min-Heap: [6, 7, 9, 10, 12, 14, 22]

Step 2 — Merge $6+7=13$: [9, 10, 12, 13, 14, 22]

Step 3 — Merge $9+10=19$: [12, 13, 14, 19, 22]

Step 4 — Merge $12+13=25$: [14, 19, 22, 25]

Step 5 — Merge $14+19=33$: [22, 25, 33]

Step 6 — Merge $22+25=47$: [33, 47]

Step 7 — Merge $33+47=80$: [80] ← Root

Assign codes by traversing tree (0=left, 1=right):

Sample Huffman Codes (tree structure dependent)

22 → 00 (shortest, most frequent)
14 → 010
19 (9+10) → 011
12 → 100
7 → 1010
6 → 1011 (longest, least frequent)

QUICK REVISION SHEET (Last-Minute Study)

Algorithms & Time Complexities

Algorithm	Best	Average	Worst
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Kruskal's MST	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$
Prim's MST	$O(V^2)$	$O(V^2)$	$O(V^2)$
Dijkstra's	$O(V^2)$	$O(V^2)$	$O(V^2)$
Bellman-Ford	$O(VE)$	$O(VE)$	$O(VE)$
Floyd-Warshall	$O(n^3)$	$O(n^3)$	$O(n^3)$
LCS (DP)	$O(mn)$	$O(mn)$	$O(mn)$
Matrix Chain (DP)	$O(n^3)$	$O(n^3)$	$O(n^3)$
TSP (DP)	$O(2^n \cdot n^2)$	$O(2^n \cdot n^2)$	$O(2^n \cdot n^2)$
Huffman Coding	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Frequently Asked Q1 Answers (2 marks each)

Question	Answer
List asymptotic notations	O (Big-Oh), Ω (Omega), Θ (Theta), o (little-oh), ω (little-omega)
Feasible vs Optimal solution	Feasible: satisfies all constraints. Optimal: feasible + maximizes/minimizes objective.
Principle of Optimality	An optimal solution contains optimal solutions to all sub-instances.
Live Node / Dead Node	Live: generated but children not yet expanded. Dead: expansion complete or killed by bound.
Non-deterministic Algorithm	Algorithm using choice() to nondeterministically select; verified in polynomial time.
P vs NP class	P: solvable in poly time. NP: verifiable in poly time. $P \subseteq NP$.
Branch and Bound	Technique that uses bounds to prune search space. Types: FIFO, LIFO, LCBB.
What is MRT?	Mean Retrieval Time. Minimized by storing programs in increasing order of length.

Negative weighted cycle	Cycle with total weight < 0 . Floyd-Warshall detects but cannot handle it correctly.
Divide and Conquer steps	Divide the problem, Conquer subproblems recursively, Combine solutions.

Exam Strategy

How to Score High (70 Marks)

Q1 (10 marks): Attempt all 6, choose best 5. Short crisp answers. Always include formal definitions.

Q2-Q5 (12 marks each): Attempt best 2 of 3. Show STEP-BY-STEP working. Draw tables for DP problems.

Q6 (12 marks): Attempt best 3 of 4. Write algorithms with proper pseudocode. State complexities.

Always asked: Asymptotic notations, LCS/MCM/TSP (DP), Greedy (Knapsack/Job Seq.), Kruskal/Prim (MST), N-Queens, m-Coloring, P vs NP.